BANCO DE DADOS

SGBD colunar em cluster, Gremlin

FGV EMAp

Júlio César Chaves

julio.cesar.chaves@prof.fgv.edu.br

Outubro, 2026

SGBD COLUNAR

As estruturas internas do SGBD foram desenhadas para operar com dados em colunas ao invés de linhas, tornando-as assim mais eficientes e otimizadas para o processamento analítico, ao invés de processamento transacional

Opera em esquema de grid, sob escalonamento linear

Os dados são armazenados e operam sob esquema de compressão



Dbdb.io (www, www2)

Lamb, A., Fuller, M., Varadarajan, R., Tran, N., Vandier, B., Doshi, L., & Bear, C. (2012). The Vertica Analytic Database: C-Store 7 Years Later. *Proceedings of the VLDB Endowment, 5*(12), 1790–1801. Retrieved from http://arxiv.org/abs/1208.4173

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

SGBD TRANSACIONAL VS ANALÍTICO

SGBDs mais voltados para transações	*Tecnologias mais voltados para análise*
Oracle	*Vertica*
SQL Server	*MongoDB*
PostgreSQL	*Splunk*
MySQL	*Hive*

Ref. PostGreSQL & Vertica: Michael Stonebraker - Wikipedia

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

O PROBLEMA É A LINGUAGEM SQL? A SOLUÇÃO É O NOSQL?

SQL

Segundo Michael Stonebraker, um dos criadores do PostgreSQL, o problema não é a linguagem SQL em si, mas sim as estruturas internas do SGBD, que inicialmente foram arquitetadas para performar transações. Michael participou da criação do Vertica, que mesmo usando SQL, performa ~50x mais rápido do que o PostgreSQL para análise de dados.

NoSQL – "Not Only SQL"

Trata-se de uma implementação para bancos de dados anamórficos, onde há plena flexibilização do modelo de dados. A estrutura vai se revelando e se atualizando ao mesmo tempo em que os dados são inseridos. A linguagem de pesquisa é expansível e detalhada, se comparada com a abstração do SQL.

Fonte: Lamb, A., Fuller, M., Varadarajan, R., Tran, N., Vandier, B., Doshi, L., & Bear, C. (2012). The Vertica Analytic Database: C-Store 7 Years Later. *Proceedings of the VLDB Endowment*, *5*(12), 1790–1801. http://arxiv.org/abs/1208.4173. Blog sobre o NoSQL do Stonebraker - https://cacm.acm.org/magazines/2011/8/114950-stonebraker-on-nosql-and-enterprises/fulltext.

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

ROW STORE VS COLUMN STORE

RowId	EmpId	Lastname	Firstname	Salary
001	10	Smith	Joe	60000
002	12	Jones	Mary	80000
003	11	Johnson	Cathy	94000
004	22	Jones	Bob	55000

Armazenamento em linha

001:10,Smith,Joe,60000;

002:12,Jones,Mary,80000;

003:11,Johnson,Cathy,94000;

004:22,Jones,Bob,55000;

Armazenamento em coluna

10:001,12:002,11:003,22:004;

Smith:001,Jones:002,Johnson:003,Jones:004;

Joe:001,Mary:002,Cathy:003,Bob:004;

60000:001,80000:002,94000:003,55000:004;

Fonte: Column-oriented DBMS – Wikipedia , Lamb, A., Fuller, M., Varadarajan, R., Tran, N., Vandier, B., Doshi, L., & Bear, C. (2012). The Vertica Analytic Database: C-Store 7 Years Later. Proceedings of the VLDB Endowment, 5(12), 1790–1801. http://arxiv.org/abs/1208.4173

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

COLUMN STORE – TIPOS DE COMPRESSÃO

	Size (MB)	Comp. Ratio	Bytes Per Row
Rand. Integers			
Raw	7.5	1	7.9
gzip	3.6	2.1	3.7
gzip+sort	2.3	3.3	2.4
Vertica	0.6	12.5	0.6
Customer Data			
Raw CSV	6200	1	32.5
gzip	1050	5.9	5.5
Vertica	418	14.8	2.2

Table 4: Compression achieved in Vertica for 1M Random Integers and Customer Data.

RLE – Run-lenght Encoding

Substitui sequências de valores idênticos por um único par que contém o valor e o número de ocorrências. Esse tipo é melhor para colunas de baixa cardinalidade.

Delta Value

Os dados são registrados como uma diferença do menor valor em um bloco de dados. Este tipo é melhor usado para colunas com muitos valores, inteiros não classificados ou baseadas em números inteiros.

Block Dictionary

Dentro de um bloco de dados, valores de coluna distintos são armazenados em um dicionário e os valores reais são substituídos por referências ao dicionário. Esse tipo é melhor para colunas não classificadas e de poucos valores, como preços de ações.

Compressed Delta Range

Armazena cada valor como um delta do anterior. Esse tipo é ideal para colunas flutuantes de muitos valores que são classificadas ou confinadas a um intervalo.

Compressed Common Delta

Constrói um dicionário de todos os deltas no bloco e, a seguir, armazena os índices no dicionário. Esse tipo é melhor para dados classificados com sequências previsíveis e quebras de sequência ocasionais. Por exemplo, instantes de data / hora registrados em intervalos periódicos ou chaves primárias.

Fonte:, Lamb, A., Fuller, M., Varadarajan, R., Tran, N., Vandier, B., Doshi, L., & Bear, C. (2012). The Vertica Analytic Database: C-Store 7 Years Later. Proceedings of the VLDB Endowment, 5(12), 1790–1801. http://arxiv.org/abs/1208.4173

Abadi MIT, D. J., Madden MIT, S. R., & Ferreira, M. C. (2006). *Integrating Compression and Execution in Column-Oriented Database Systems.*

Copyright (c) 2020 Nenad Jukic and Prospect Press

FUNÇÕES ANALÍTICAS EM SQL

O que são?

São funções SQL baseadas no padrão SQL ANSI 99. Essas funções lidam com análises e relatórios complexos – tais como:

Ranqueamento dos clientes mais antigos de um determinado estado.
Calcular a média móvel do volume de vendas à varejo em um período específico.
Encontrar a pontuação mais alta entre todos os alunos da mesma série.
Comparar o valor do bônus atual de vendas com o valor recebido no bônus anterior.
Obter a matriz de origem-destino de mobilidade a partir de dados de telefonia móvel.

Fonte : Documentação do Vertica

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

Funções analíticas em SQL

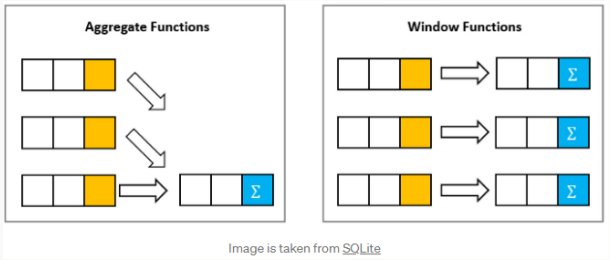


Image is taken from SQLite

Imaginar "sub-grupos" de GROUP BY

Sabemos que o GROUP BY leva em conta a tabela inteira. Agora imagine que você pode particionar a tabela e sub-agregar os dados.

Como usar?

A sintaxe geral é disposta da seguinte forma:

analytic-function(arguments) OVER(

[window-partition-clause]

[window-order-clause [window-frame-clause]]

)

Sendo:

analytic-function(arguments) - uma função interna do SGBD, no caso o Vertica da Microfocus.

OVER – especifica como particionar, ordenar, e atribuir funções de janelas dinâmicas sobre os dados de entrada. Os dados de entrada são aqueles que retornam de uma consulta depois de validar as cláusulas FROM, WHERE, GROUP BY, e HAVING.

window-partition-clause – agrupa as linhas de acordo com uma ou mais colunas ou expressões. Se for omitido, não haverá agrupamento e a função analítica irá processor todas as linhas de entrada como uma partição única.

window-order-clause – opcionalmente especifica como ordenar as linhas que vêm da função analítica. Se o OVER incluir uma cláusula de partição, as linhas são ordenadas respeitando cada partição.

window-frame-clause – válida apenas para algumas funções analíticas, especifica como entrada um conjunto de linhas relativas à linha corrente. Depois que a função processa essa linha e sua respectiva janela, avança a linha corrente e ajusta os limites da janela de acordo com a nova linha corrente.

Fonte : Documentação do Vertica, www2

SQL Server e as funções analíticas

```
Use DWZAGI;

-- RANQUER produtos que mais venderam

SELECT  NomeProduto, SumVendas,
        PERCENT_RANK() OVER(ORDER BY SumVendas) AS "PERCENT_RANK()",
        RANK() OVER(ORDER BY SumVendas) AS "RANK()"
FROM
    (select
        sum(v.ReaisVendidos) as SumVendas, p.NomeProduto
    FROM
        vendas v inner join produto p on v.ChaveProduto = p.ChaveProduto
    GROUP By
        p.NomeProduto) "SubSelect1";
```

21 %

Results | Messages

	NomeProduto	SumVendas	PERCENT_RANK()	RANK()
1	Leite Integral Piracanjuba 1L	3.99	0	1
2	BAW WAW AREIA SANITÁRIA PERFUMADA PARA GATOS 4kg	6.5	0.0555555555555556	2
3	Multilaser Luva Em Microfibra Dupla Face Limpeza E Polimento 1...	12.9	0.111111111111111	3
4	Milagre Creme Pentear 450G, Lola Cosmetics	20.9	0.166666666666667	4
5	Limpa e Hidrata Couro Proauto 150 g	22.52	0.222222222222222	5
6	Tapete Higiênico Tico Pads, 30 Unidades	44.52	0.277777777777778	6
7	Toalha De Microfibra Vonder 45 X 38 Cm	54.66	0.333333333333333	7
8	Licor Amarula 375 ml	62.9	0.388888888888889	8
9	Composto Lácteo Milnutri Premium Danone Nutricia 400g	91.78	0.444444444444444	9
10	Máscara Super Hidratante Morte Súbita, Lola Cosmetics, 450g	98.7	0.5	10
11	Liquidificador Mini portátil USB,verde	116.1	0.555555555555556	11
12	Coleira Antipulgas e Carrapatos Bayer Seresto para Cães e Gatos...	339.56	0.611111111111111	12
13	Aparador de Pelos Mondial, Super Groom - 06, Preto/Verde, Bivo...	359.96	0.666666666666667	13
14	Depurador de ar Slim branco com manta 80cm 110V Suggar	809.7	0.722222222222222	14
15	Frigobar, 45L, Branco, 110v, Midea	1498	0.777777777777778	15
16	Adega Brastemp 12 Garrafas com painel touch - 110V	1598	0.833333333333333	16
17	Apoquel 16Mg, 20Compr Zoetis Para Cães	1799.67	0.888888888888889	17
18	Fogão 5 bocas Dako Supreme Timer Glass Titanium com mesa d...	3398	0.944444444444444	18
19	Hugo Boss Bottled Eau De Toilette 30Ml	4761	1	19

Analytic Functions (vertica.com)

Fonte: www1

A TABELA DE TRANSAÇÕES DE VENDA

Results | Messages

	tid	customerid	storeid	tdate
1	10.027	1-2-333	S2	2021-01-09
2	10.0576	1-2-333	S3	2020-10-16
3	10.0786	1-2-333	S1	2021-04-15
4	10.1234	1-2-333	S1	2020-05-18
5	10.273	2-3-444	S1	2022-02-24
6	10.3753	2-3-444	S3	2022-10-03
7	100.825	3-4-555	S2	2022-11-08
8	1000.2	1-2-333	S1	2022-11-09
9	1000.4	1-2-333	S3	2020-11-28
10	1000.47	3-4-555	S2	2020-10-25

A tabela de transações de venda está assim.

1. Contar transações por dia e por loja;

2. Adicionar a média de transações por loja para todos os dias;

3. Listar em ordem descendente de data.

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

```sql
-- A partir de quantidade de transações por loja e por dia,
-- acrescentar uma coluna que mostre a média de transações
-- ocorridas naquela loja em todos os dias da base
-- em ordem descendente de dia

select qtd_trn, AVG(qtd_trn) OVER (PARTITION BY storeid) as avg_loja,
storeid, tdate
FROM
(select count(*) as qtd_trn, storeid, tdate
from salestransaction
group by tdate, storeid) "s1"
order by tdate desc;
```

PostgreSQL: Documentation: 16: 3.5. Window Functions

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

SQL analítico para inferir mobilidade

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

DataHora	duracao	ID1	LATITUDE	LONGITUDE	ID2
2014-01-09 09:42:28	1.12	21CB815B61E822D7792E2049C96933F1CC	-22.78465	-43.30725	2182CA6E7D073E82CE136FC59D2FDBA168
2013-12-09 17:28:51	0.2	21A6AC61F7091B28835796D96948065551	-22.79742	-43.31212	2182CA6E7D073E82CE136FC59D2FDBA168
2013-12-09 09:13:34	1.57	21D5E24F7E040117534E73D6C198E7D517	-22.7961	-43.2979	2182CA6E7D073E82CE136FC59D2FDBA168
2014-04-09 16:17:44	0.48	216A8313A3921330D0DEEBFC3C30FC8BF9	-22.78893	-43.30631	2182CA6E7D073E82CE136FC59D2FDBA168
2014-04-09 16:07:40	1.32	216A8313A3921330D0DEEBFC3C30FC8BF9	-22.7961	-43.2979	2182CA6E7D073E82CE136FC59D2FDBA168
2014-04-09 09:48:01	0.22	214038B104297308F291FAAACA707C53FE	-22.78893	-43.30631	2182CA6E7D073E82CE136FC59D2FDBA168
2014-04-09 10:10:28	1	21CB815B61E822D7792E2049C96933F1CC	-22.78465	-43.30725	2182CA6E7D073E82CE136FC59D2FDBA168
2014-05-09 15:32:19	0.23	211FC2D559C0CCB62370AB70A17EDE2882	-22.78893	-43.30631	2182CA6E7D073E82CE136FC59D2FDBA168
2014-05-09 15:12:40	0.65	211FC2D559C0CCB62370AB70A17EDE2882	-22.7775	-43.3156	2182CA6E7D073E82CE136FC59D2FDBA168
2014-05-09 10:38:40	2.15	213F7D452C93453FE06FC295D5827D551D	-22.77671	-43.2939	2182CA6E7D073E82CE136FC59D2FDBA168
2014-06-09 14:06:25	2.27	21B182A31004C20F5A2C084390251198CD	-22.758	-43.3191	2182CA6E7D073E82CE136FC59D2FDBA168
2014-02-09 22:24:42	2.03	210976172F2F5C34BD30687324222E15C4	-22.7112	-43.2618	2182CF4F56CA8AAD996AF1E5DF13757330
2014-05-09 19:04:08	18.02	21C9788C5BA0F02D44805BF88E5325671A	-22.74891	-43.28878	2182D08C8D562145A7245E2817ABC99EEA
2013-12-09 11:38:58	0.57	211A731DE94D589CD5FE5A99A65503B2BF	-22.77671	-43.2939	2182D9BF63FC7CC9E20CD8EC7195F6BEB6
2014-03-09 10:32:48	3.37	21C92049958DF9441CF004F8E306B437C7	-22.78893	-43.30631	2182D9BF63FC7CC9E20CD8EC7195F6BEB6
2014-03-09 10:27:28	5.03	21C92049958DF9441CF004F8E306B437C7	-22.78893	-43.30631	2182D9BF63FC7CC9E20CD8EC7195F6BEB6
2014-02-09 15:11:40	1.05	21A83CAC2C59CFEA98AF71FBA2F598EB62	-22.758	-43.3191	2182E50D9F1AE1CBFDBFCBA06A3D852746
2014-07-09 12:24:13	0.42	21386E6808E0190D78BA0E83E517EAC274	-22.78611	-43.31112	2182E5DCE0858ED1B42BDAB5F8D57577FE

Fonte: Chaves, J. C. (2018). *ESTIMATIVA DE MATRIZES DE ORIGEM-DESTINO A PARTIR DE REGISTROS DE CHAMADAS DE TELEFONIA CELULAR*. UFRJ.

```
select
  ddd_orig||ID1 as id,
  upper(b.RA) as local,
  b.LAT||','||b.LNG as coord,
  DT_INI_ATIVIDADE_REDE||'_'||HR_INI_ATIVIDADE_REDE as datahora,
  HR_INI_ATIVIDADE_REDE as hora,
  conditional_change_event(b.LAT||','||b.LNG) over
    (partition by ddd_orig||ID1 order by ddd_orig||ID1,DT_INI_ATIVIDADE_REDE||'_'||HR_INI_ATIVIDADE_REDE) as salto
from
  bill_QUANDO_ONDE_J  a inner join BDU_DW_150306074501 b on cast(b.cell_id as int) =  a.cell_id_orig
where DT_INI_ATIVIDADE_REDE = '2014-01-01'
   and b.CN='21'
order by ddd_orig||ID1, salto,
    DT_INI_ATIVIDADE_REDE||'_'||HR_INI_ATIVIDADE_REDE,
    b.RA
    ) as salto
```

Fonte: Chaves, J. C. (2018). *ESTIMATIVA DE MATRIZES DE ORIGEM-DESTINO A PARTIR DE REGISTROS DE CHAMADAS DE TELEFONIA CELULAR*. UFRJ.

```
select id, datahora,
  salto, hora,
  lead(hora,1) over (partition by id order by datahora) as nexthour,
  local as Origem ,
  lead(local,1) over (partition by id order by datahora) as Destino,
  coord as OCoord,
  lead(coord,1) over (partition by id order by datahora) as DCoord
from (
```

```
    )as grouped
```

Fonte: Chaves, J. C. (2018). *ESTIMATIVA DE MATRIZES DE ORIGEM-DESTINO A PARTIR DE REGISTROS DE CHAMADAS DE TELEFONIA CELULAR*. UFRJ.

```
select id, datahora, salto, Origem , Destino,
  OCoord,
  DCoord,
  round(distance(
  cast(substr(OCoord,0,instr(OCoord,','))as float) ,
  cast(substr(OCoord,instr(OCoord,',')+1,length(OCoord))as float) ,
  cast(substr(DCoord,0,instr(DCoord,','))as float) ,
  cast(substr(DCoord,instr(DCoord,',')+1,length(DCoord))as float) ),3) as km_distance,
  datediff(mi,hora, nexthour) as minsbetween
from (
```

```
    )as grouped2
```

Fonte: Chaves, J. C. (2018). *ESTIMATIVA DE MATRIZES DE ORIGEM-DESTINO A PARTIR DE REGISTROS DE CHAMADAS DE TELEFONIA CELULAR*. UFRJ.

```
select id, datahora, salto, Origem , Destino, OCoord, DCoord, minsbetween
from (
```

```
where OCoord != DCoord
  and km_distance > 2
  and minsbetween > 30
  and minsbetween < 240
  ) as grouped3
```

Fonte: Chaves, J. C. (2018). *ESTIMATIVA DE MATRIZES DE ORIGEM-DESTINO A PARTIR DE REGISTROS DE CHAMADAS DE TELEFONIA CELULAR*. UFRJ.

```
select '2014-01-01' as dia ,Origem , Destino, count(*)
from (
```

```
  ) as grouped3
GROUP BY Origem, Destino;
```

Fonte: Chaves, J. C. (2018). *ESTIMATIVA DE MATRIZES DE ORIGEM-DESTINO A PARTIR DE REGISTROS DE CHAMADAS DE TELEFONIA CELULAR*. UFRJ.

```
select '2014-01-01' as dia ,Origem , Destino, count(*)
from (
select id, datahora, salto, Origem , Destino, OCoord, DCoord, minsbetween
from (
select id, datahora, salto, Origem , Destino,
  OCoord,
  DCoord,
  round(distance(
  cast(substr(OCoord,0,instr(OCoord,','))as float) ,
  cast(substr(OCoord,instr(OCoord,',')+1,length(OCoord))as float) ,
  cast(substr(DCoord,0,instr(DCoord,','))as float) ,
  cast(substr(DCoord,instr(DCoord,',')+1,length(DCoord))as float) ),3) as km_distance,
  datediff(mi,hora, nexthour) as minsbetween
from (
select id, datahora,
  salto, hora,
  lead(hora,1) over (partition by id order by datahora) as nexthour,
  local as Origem ,
  lead(local,1) over (partition by id order by datahora) as Destino,
  coord as OCoord,
  lead(coord,1) over (partition by id order by datahora) as DCoord
from (
select
  ddd_orig||ID1 as id,
  upper(b.RA) as local,
  b.LAT||','||b.LNG as coord,
  DT_INI_ATIVIDADE_REDE||'_'||HR_INI_ATIVIDADE_REDE as datahora,
  HR_INI_ATIVIDADE_REDE as hora,
  conditional_change_event(b.LAT||','||b.LNG) over
    (partition by ddd_orig||ID1 order by ddd_orig||ID1,DT_INI_ATIVIDADE_REDE||'_'||HR_INI_ATIVIDADE_REDE) as salto
from
  bill_QUANDO_ONDE_J  a inner join BDU_DW_150306074501 b on cast(b.cell_id as int) =  a.cell_id_orig
where DT_INI_ATIVIDADE_REDE = '2014-01-01'
   and b.CN='21'
order by ddd_orig||ID1, salto,
    DT_INI_ATIVIDADE_REDE||'_'||HR_INI_ATIVIDADE_REDE,
    b.RA
    ) as salto
    )as grouped
    )as grouped2
where OCoord != DCoord
  and km_distance > 2
  and minsbetween > 30
  and minsbetween < 240
  ) as grouped3
GROUP BY Origem, Destino;
```

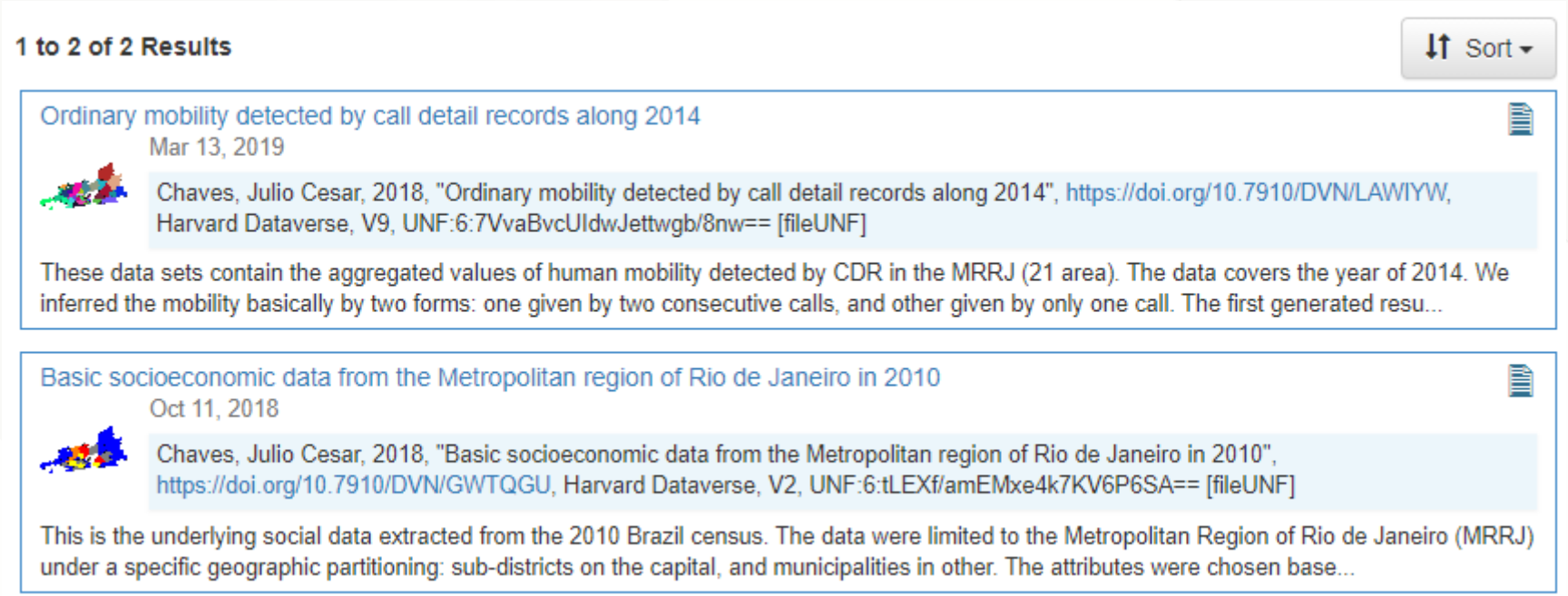
Fonte:

DADOS PÚBLICOS NO DATAVERSE

Metropolitan Region of Rio de Janeiro (harvard.edu)

https://dataverse.fgv.br/dataverse/RJMA

J. C. Chaves, M. A. H. B. da Silva, R. de S. Alencar, A. G. Evsukoff, and V. da F. Vieira, "Human mobility and socioeconomic datasets of the Rio de Janeiro metropolitan area," *Data Br.*, vol. 51, 2023, doi: 10.1016/j.dib.2023.109695.



Fonte: Chaves, J. C. (2018). *ESTIMATIVA DE MATRIZES DE ORIGEM-DESTINO A PARTIR DE REGISTROS DE CHAMADAS DE TELEFONIA CELULAR*. UFRJ.

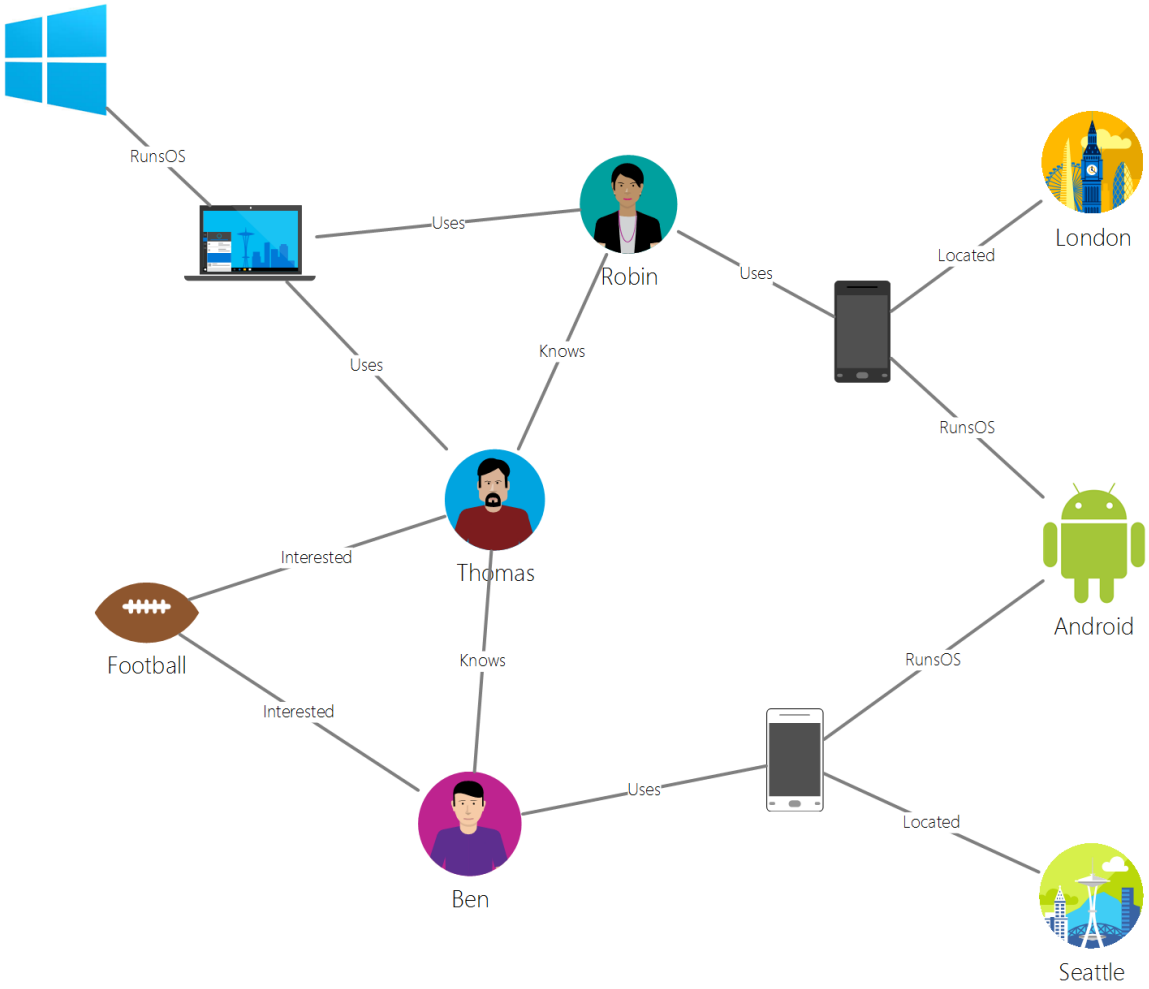
Copyright (c) 2020 Nenad Jukic and Prospect Press

Bases de dados em grafos

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

EXEMPLO DE GRAFO



Este grafo possui quatro tipos de ***vértices***:
Pessoas: Robin, Thomas, e Ben
Interesses: O interesse deles, por exemplo futebol.
Dispositivos.
Sistemas Operacionais.
As relações são representadas pelas ***arestas***:
Knows: Por exemplo, "Thomas conhece Robin"
Interesse: Representa o interesse da pessoas, por exemplo, "Ben é interessado em futebol"
RunsOS: Laptop roda o Windows OS
Uses: Representa qual dispositivo pertence a qual usuário.

https://docs.microsoft.com/en-us/azure/cosmos-db/graph-introduction

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

ALGUMAS TECNOLOGIAS PARA GRAFOS

Cambridge Semantics

Pode-se usar o DB AnzoGraph gratuito até 16GB (www).

Neo4j

Plataforma de DB para grafos com sandbox (www).

Gremlin

Proposta de uma linguagem universal para bancos de dados de grafos. (www)

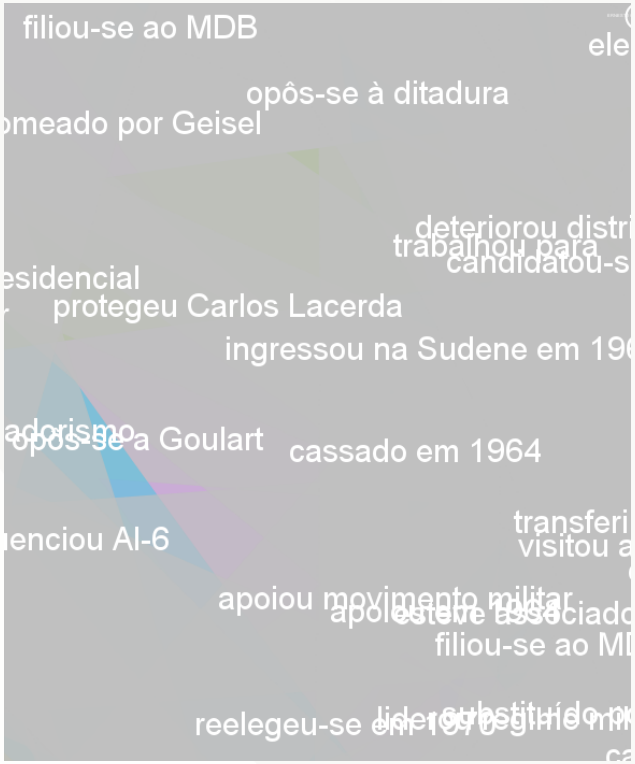
Graphileon

Plataforma de visualização de grafos. (www)

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

RAG, GRAPHRAG, EX. DHBB



RAG - Leitura aumentada por IA

Exemplo ChatFGV;

DH como corpus

https://github.com/Escola-de-Matematica-Aplicada/ChatFGV.git

GraphRAG

Percorrer os pedaços de texto em busca de conexões

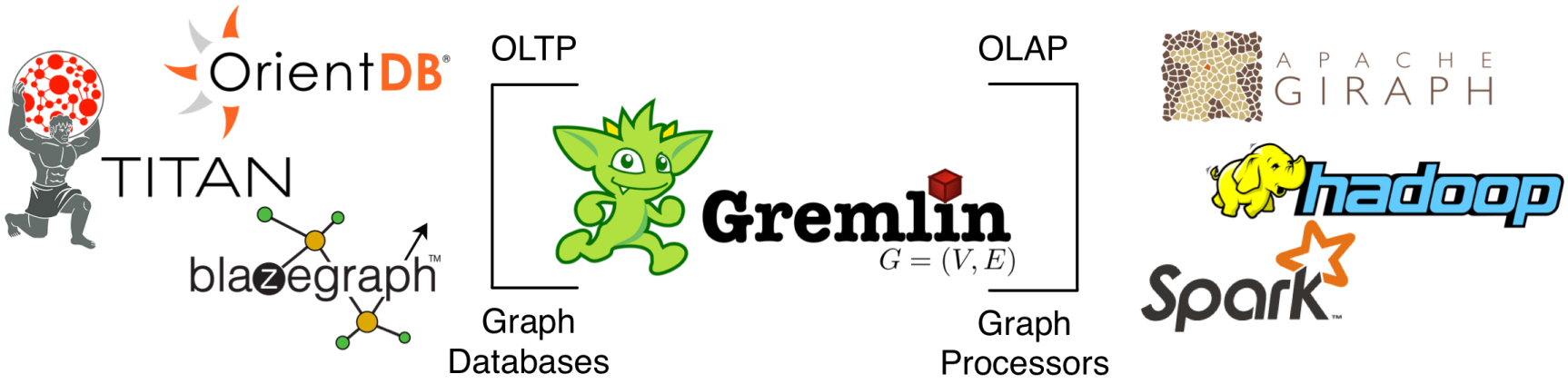
https://www.microsoft.com/en-us/research/project/graphrag/

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

APACHE TINKERPOP GREMLIN

- Linguagem para bancos de dados orientados a grafos (*graph traversal machine*)
- É composta por três componentes: *G* (dados), Ψ (instruções), e um conjunto de caminhos *T*
- Respeita a linguagem nativa dos ambientes OLTP e OLAP
- O código Gremlin pode rodar em diversas plataformas (Spark, TITAN, Cosmos DB)



Rodriguez, M. A. (2015). The gremlin graph traversal machine and language (Invited Talk). *DBPL 2015 - Proceedings of the 15th Symposium on Database Programming Languages*, 1–10. https://doi.org/10.1145/2815072.2815073

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

LIGUAGEM GREMLIN: DEFINIÇÕES

Gremlin é uma linguagem universal aplicada para percorrer grafos.

A linguagem funciona a partir de três componentes básicos:

(1)

Um grafo

O grafo precisa ser direcionado (um dígrafo), consistindo em um conjunto não vazio de vértices e de arestas direcionadas, aonde cada aresta está associada a um par ordenado de vértices, chamados "terminações" (endpoints). Se a aresta *a* está associada a um par *(v,w)* de vértices, então diz-se que a aresta *a* é direcionada de *v* para *w*.

(2)

Um conjunto de "traversers"

Conceitualmente, uma coleção de "traversers" ***T*** move-se sobre o grafo ***G*** de acordo com as instruções ψ.

A computação é finalizada quando:

1) Não há *traverser* em ***T***, ou
2) Todos os *traversers* ***T*** não são mais referenciados por instruções em ψ (i.e., foram até as terminações).

No caso (1), o resultado é um conjunto vazio.

No caso (2), o resultado é o conjunto união das localizações sobre ***G*** percorridas pelos traversers ***T***.

"Traverser" é o processo de percorrer o grafo em si (na falta de um termo equivalente, mantivemos traverser).

(3)

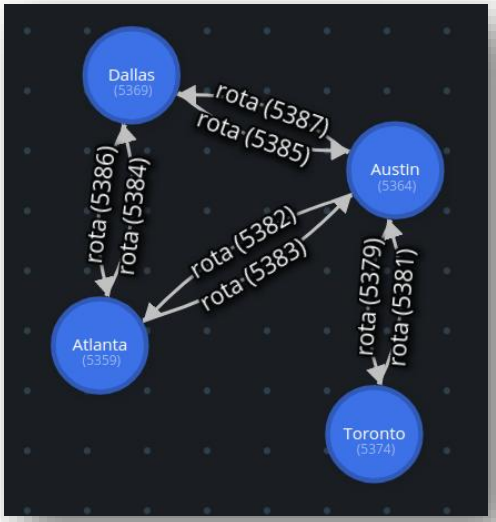
Um conjuno de instruções de travessia

Fonte: Rodriguez, M. A. (2015). The gremlin graph traversal machine and language (Invited Talk). DBPL 2015 - Proceedings of the 15th Symposium on Database Programming Languages, 1–10. https://doi.org/10.1145/2815072.2815073.

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

GREMLIN: COMANDOS BÁSICOS



Início

Uma consulta em um grafo é geralmente chamada de travessia, pois a consulta irá percorrer o grafo.

As travessias consistem em uma ou mais etapas no Gremlin, que são basicamente métodos encadeados.

Praticamente todos os comandos de travessia no Gremlin iniciam com g.V() ou g.E(), indicando que serão aplicadas instruções sobre vértices ou arestas, respectivamente.

Os comandos mais básicos permitem escolher determinados vértices de acordo com o tipo do vértice e critérios de “chave-valor”:

g.V().hasLabel(‘aeroporto’) – irá listar o código de todos os vértices que representam aeroportos.

g.V().has(‘código’,’SDU’) - irá listar o código interno (ID) do aeroporto SDU.

Escolha de atributos

Os comandos mostrados anteriormente irão listar apenas os códigos internos dos vértices. Para listar os valores pertencentes aos vértices, é necessário adicionar .values(), conforme a seguir.

g.V().has(‘código’,’SDU’).

values('cidade') - irá listar a cidade onde se encontra o Aeroporto SDU.

g.V().has(‘código’,’SDU’).

values() - irá listar todos os atributos do vértice que identifica o Aeroporto SDU.

Contagem de vértices

O comando g.V().hasLabel('aeroporto'). outE('rota').count(), irá contar os aeroportos que possuem rotas para outros aeroportos.

Fonte: https://www.kelvinlawrence.net/book/Gremlin-Graph-Guide.html#gremlinandsql

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

COMPARATIVO ENTRE O GREMLIN E OUTRAS LINGUAGENS

Se-Então-Senão vs. Gremlin

No exemplo abaixo, se o vértice for do tipo "Pessoa", então contar a quantidade de arestas saindo, o que equivale a quantidade de projetos. Se o vértice for do tipo "Software", então contar a #arestas entrando, o que equivale a #Pessoas participando do projeto.

Se-Então-Senão

```
Se Vértice = "Pessoa" Então
    Contar quantidade de \
        projetos criados;
Senão Se Vértice = "Software" Então
    Contar quantidade de \
        colaboradores que estão
        participando do projeto
        de software
Senão
    Retona "vazio"
```

Linguagem Gremlin

```
g.V().choose(label()).
option("Pessoa",out("criou").count()).
option("Software",in("criou").count()).
option(none.label())
```

SQL vs. Gremlin

ID	CODE	ICAO	CITY	COUNTRY
1	ATL	KATL	Atlanta	US
3	AUS	KAUS	Austin	US
8	DFW	KDFW	Dallas	US
47	YYZ	CYYZ	Toronto	CA
49	LHR	EGLL	London	UK
51	CDG	LFPG	Paris	FR
52	FRA	EDDF	Frankfurt	DE
55	SYD	YSSY	Sydney	AU

No exemplo abaixo, contar a quantidade de aeroportos por país. Em SQL à esquerda e em Gremlin à direita.

```
select
    country,count(country)
from
    airports
group by
    country;
```

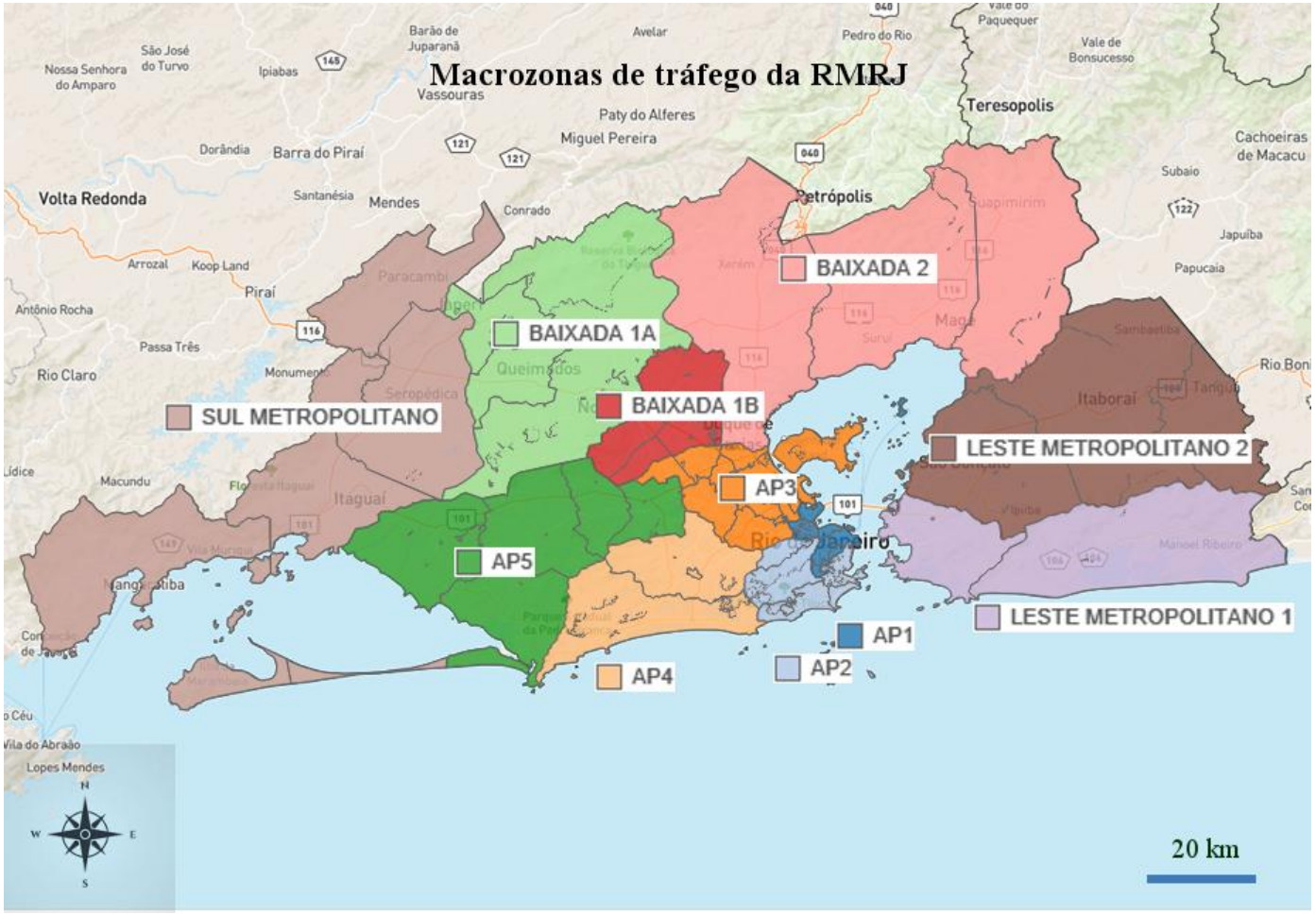
```
g.V().hasLabel('airport').
groupCount().by('country')
```

Fonte: https://www.kelvinlawrence.net/book/Gremlin-Graph-Guide.html#gremlinandsql

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

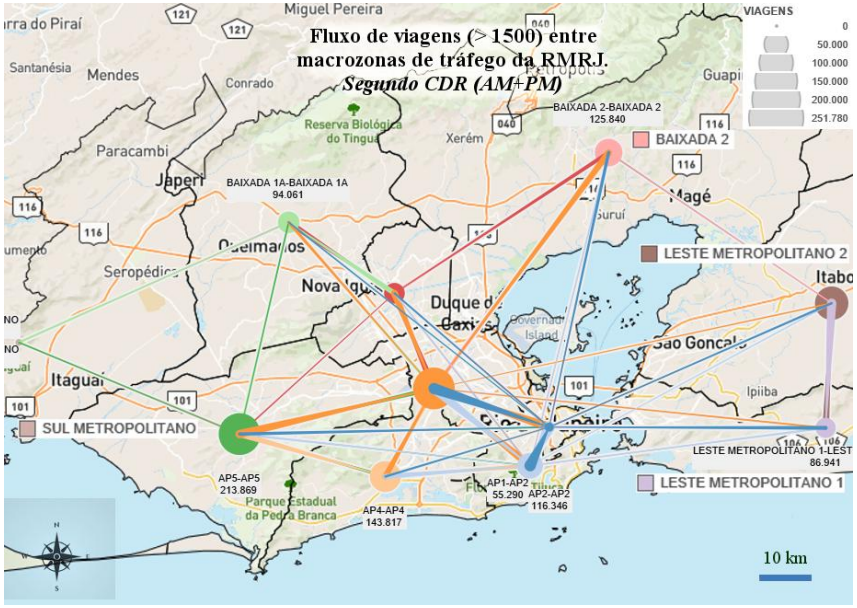
EXEMPLO DE USO
MACROZONAS DE TRÁFEGO DA REGIÃO METROPOLITANA DO RJ



© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

DESAFIO DE VISUALIZAÇÃO PARA GRAFO PLANAR






```
drop table if exists tmp_caminho;
CREATE TEMPORARY TABLE tmp_caminho as
WITH recursive transitive_closure (origem, destino, distancia, saltos, path_string) AS
(  SELECT origem, destino, distancia, 1 as saltos,
      '.' || origem || '.' || destino || '.' AS path_string
  FROM cdr.mz_arestas
  WHERE origem = 'LESTE METROPOLITANO 1' –source   UNION ALL
  SELECT tc.origem, e.destino, tc.distancia + e.distancia, tc.saltos + 1,  tc.path_string || e.destino || '.' AS
path_string FROM cdr.mz_arestas as e  JOIN transitive_closure AS tc ON e.origem = tc.destino
  WHERE tc.path_string NOT LIKE '%.' || e.destino || '.%' )
SELECT origem, destino, path_string, distancia, saltos from transitive_closure
WHERE destino='SUL METROPOLITANO' --destino
ORDER BY origem, destino, distancia;

SELECT origem, destino, path_string, distancia, saltos from tmp_caminho
WHERE distancia = (SELECT MIN(distancia) from tmp_caminho);
```

```
g.V('LESTE METROPOLITANO
1').repeat(out().simplePath()).until(hasId('SUL
METROPOLITANO')).path().limit(1)
```

© Duarte, Inc. 2014

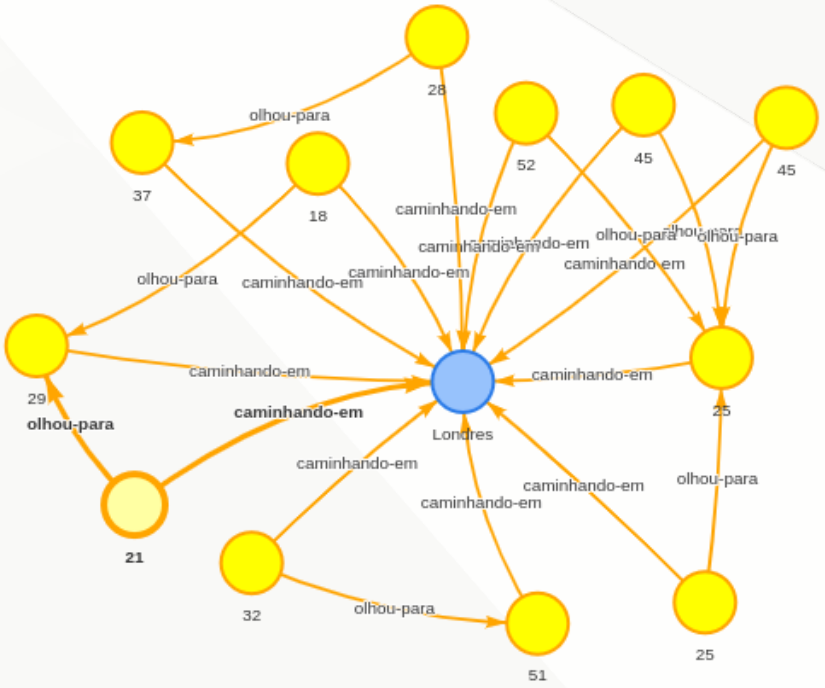
Copyright (c) 2020 Nenad Jukic and Prospect Press

RANQUEAMENTO DE OLHARES

Descrição do problema

Como ranquear os olhares trocados entre pessoas que passam na rua Oxford em frente à Microsoft em Londres.

Imaginemos que os dados foram coletados por alguns segundos e pré-processados a partir das imagens das câmeras de observação.



Cada pessoa caminhando na Rua Oxford em Londres em frente a Microsoft no dia 4/7/21 às 8:31:23 é representada por um vértice amarelo.

O local específico em Londres é o vértice azul.

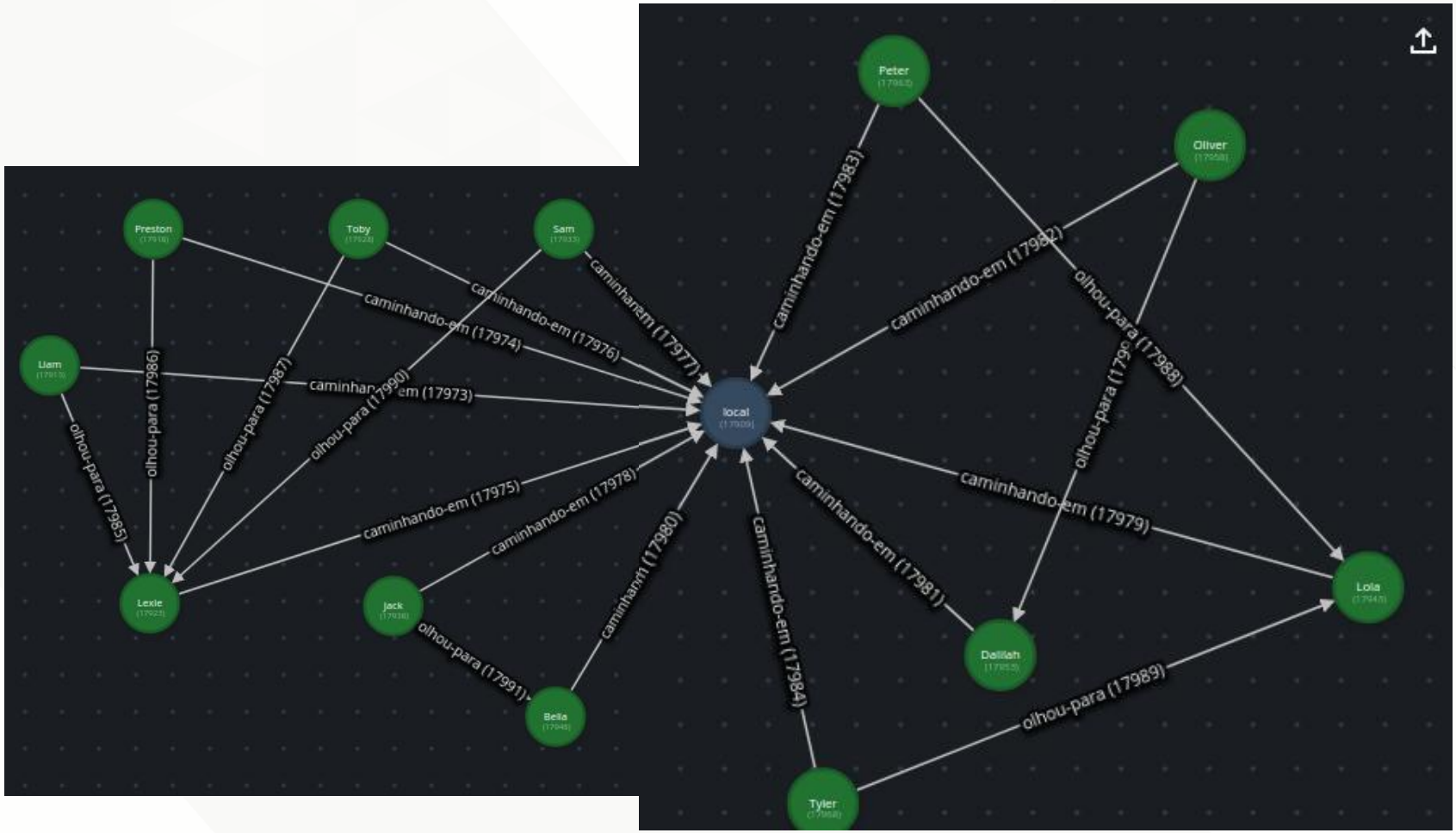
Existem dois tipos de arestas: “olhou-para”, indicando que uma pessoa olhou para outra no sentido da seta; e “caminhando-em”, indicando que a pessoa estava caminhando naquele local.

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

O GRAFO

Visualização do grafo com os nomes fictícios das pessoas (vértices verdes) em Londres (vértice ao centro).

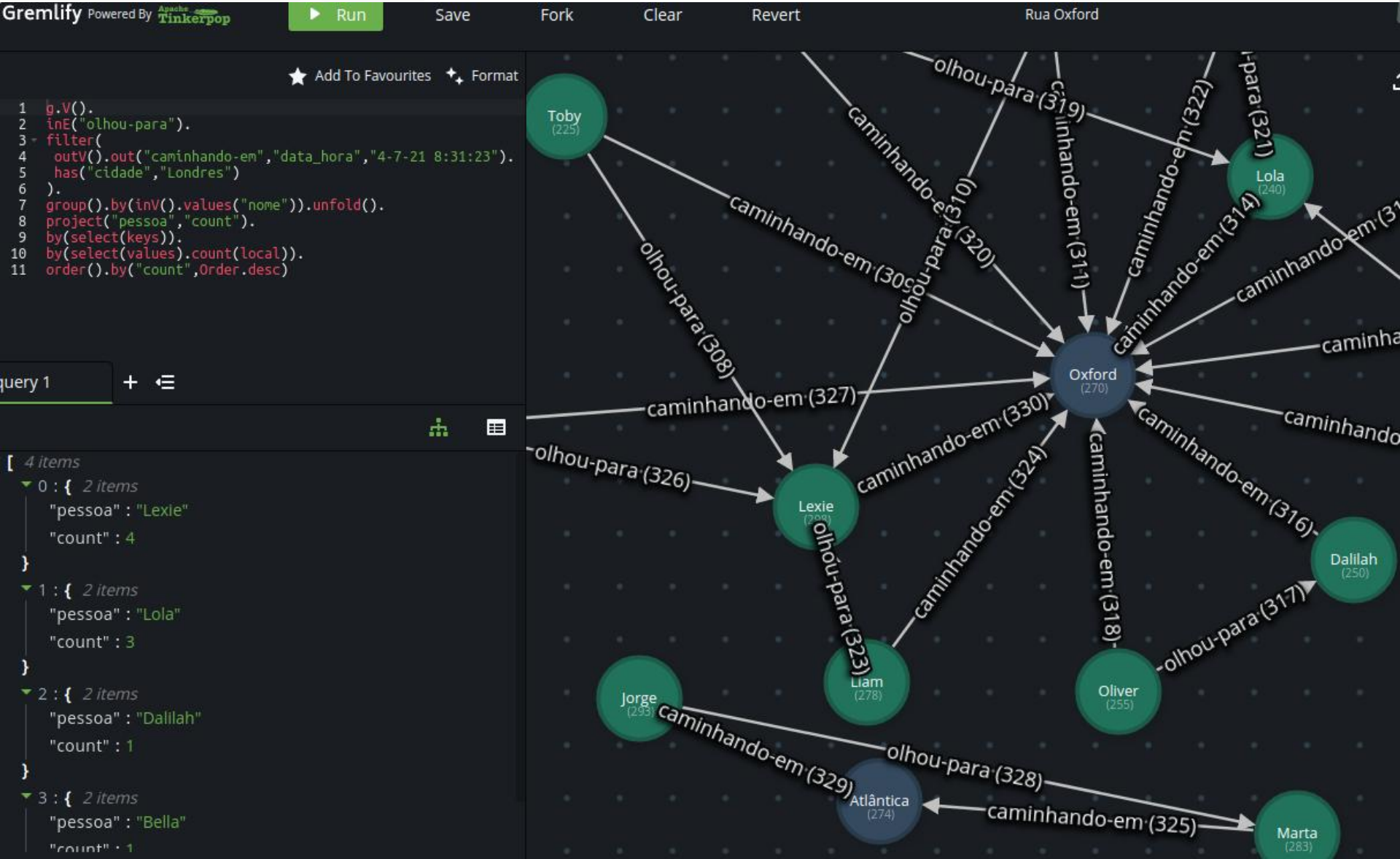


© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

GREMLIFY.COM

Projeto de software livre para criar grafos visualmente e testar comandos Gremlin. O resultado está à esquerda inferior.



A CONSULTA

```
g.V().
inE("olhou-para").
filter(
 outV().out("caminhando-em",
"data_hora","4-7-21 8:31:23").
 has("cidade","Londres")
).
group().by(inV().
values("nome")).unfold().
project("pessoa","count").
by(select(keys)).
by(select(values).count(local)).
order().by("count",Order.desc)
```

Selecionar todos os vértices que recebem arestas "olhou-para", que representam todas as pessoas que receberam olhares.

Filtrar apenas pessoas que estavam caminhando em Londres em 4 de julho de 2021 às 8:31:23.

Agrupar pelos vértices que recebem arestas (pessoas que recebem olhares) e projetar o nome da pessoa e a quantidade de olhares.

Ordenar do maior ao menor, pela quantidade de olhares recebidos. O primeiro registro será o da pessoa que recebeu mais olhares.

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press

Exercício de laboratório

A partir do grafo dos deuses gregos, responda as seguintes questões sobre os deuses.

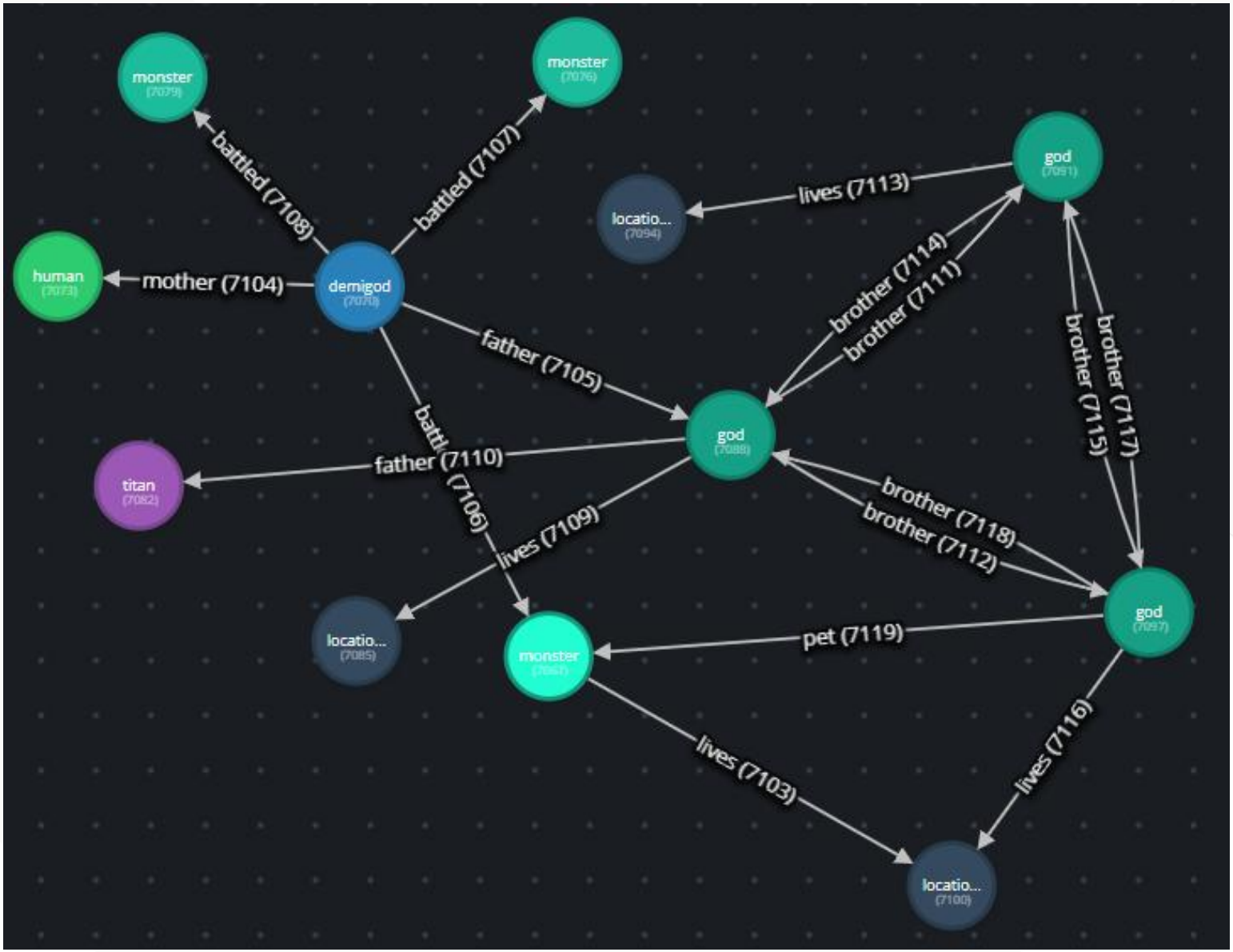
Com quem Hercules lutou?

Quem eram os pais de Hercules?

Qual o deus que vivia no mar?

© Duarte, Inc. 2014

Copyright (c) 2020 Nenad Jukic and Prospect Press



ego network greek mythology.txt

CONSIDERAÇÕES FINAIS

Limitações de uma nova tecnologia

Apesar de uma comunidade ativa de software livre, a linguagem Gremlin possui limitações em relação ao tratamento de erros. Assim sendo, o usuário precisa compreender muito bem a linguagem antes de conseguir usar, além do domínio dos conceitos de grafo serem um pré-requisito. A visualização com software livre tem evoluido.

Simplicidade de uso

O usuário que dominar a linguagem Gremlin poderá executar com relativa simplicidade as operações de consulta em uma base de dados de grafo.

Basicamente, a linguagem Gremlin foi estruturada para operar sobre um dígrafo $G=(V,E,\lambda)$, sendo λ o conjunto de instruções desejadas para percorrer o grafo G através dos vértices contidos em V e/ou arestas contidas em E.

Fontes: https://docs.microsoft.com/en-us/azure/cosmos-db/graph-visualization.

Copyright (c) 2020 Nenad Jukic and Prospect Press

COMANDO DE CRIAÇÃO DO GRAFO DO EXEMPLO

Clicar no ícone abaixo para abrir o comando de criação do grafo usado no exemplo de ranqueamento de olhares.



Grafo para testar ranqueamento de beleza.txt

Copyright (c) 2020 Nenad Jukic and Prospect Press